

NAME: SINGIREDDY AMULYA REDDY

REGNO: 20233273

SEC: D

NAME: SINGIREDDY AMULYA REDDY

REGNO: 20233273

SEC: D

```
import numpy as np
from collections import defaultdict

class NaiveBayesClassifier:
    def __init__(self):
        self.priors = {}
        self.likelihoods = {}
        self.classes = set()

    def fit(self, X_train, y_train):
        # Calculate prior probabilities
        self.classes = set(y_train)
        for c in self.classes:
            self.priors[c] = np.mean(np.array(y_train) == c)

        # Calculate likelihoods
        word_counts = {c: defaultdict(int) for c in self.classes}
        total_words = {c: 0 for c in self.classes}

        for email, label in zip(X_train, y_train):
            for word, count in email.items():
                word_counts[label][word] += count
                total_words[label] += count

        for c in self.classes:
            self.likelihoods[c] = {}
            for word in word_counts[c]:
                self.likelihoods[c][word] = (word_counts[c][word] + 1) / (
                    total_words[c] + len(word_counts[c])
                )

    def predict(self, X_test):
        predictions = []
        for email in X_test:
            posteriors = {}

            for c in self.classes:
                posteriors[c] = np.log(self.priors[c])

                for word, count in email.items():
                    if word in self.likelihoods[c]:
                        posteriors[c] += np.log(self.likelihoods[c][word]) * count
                    else:
                        # Laplace smoothing for unseen words
                        posteriors[c] += np.log(
                            1 / (sum(len(self.likelihoods[c]) for c in self.classes) + len(self.likelihoods[c]))
                        ) * count

            predictions.append(max(posteriors, key=posteriors.get))
        return predictions

    def evaluate(self, X_test, y_test):
        predictions = self.predict(X_test)
        accuracy = np.mean(np.array(predictions) == np.array(y_test))
        return accuracy

if __name__ == "__main__":
    dataset = [
        ({'offer': 3, 'win': 2, 'prize': 1, 'click': 4, 'free': 2}, 'spam'),
        ({'hello': 2, 'meeting': 3, 'schedule': 1, 'project': 5, 'team': 4}, 'ham'),
        ({'money': 5, 'transfer': 3, 'account': 2, 'urgent': 4, 'link': 1}, 'spam'),
        ({'assignment': 3, 'deadline': 2, 'submit': 4, 'professor': 1, 'university': 2}, 'ham'),
        ({'discount': 2, 'limited': 3, 'offer': 5, 'buy': 4, 'now': 6}, 'spam'),
        ({'family': 3, 'trip': 4, 'vacation': 2, 'plans': 5, 'tickets': 6}, 'ham'),
        ({'lottery': 5, 'winner': 2, 'prize': 3, 'congratulations': 1, 'claim': 4}, 'spam'),
        ({'meeting': 1, 'agenda': 2, 'discuss': 3, 'tasks': 4, 'workshop': 5}, 'ham'),
        ({'investment': 4, 'profits': 3, 'opportunity': 5, 'risk-free': 2, 'money': 6}, 'spam'),
    ]
```

```
    ({"friend": 2, "birthday": 5, "party": 3, "invite": 4, "venue": 1}, "ham")  
]
```

```
# Split dataset (80% training, 20% testing)  
split_idx = int(0.8 * len(dataset))  
train_data, test_data = dataset[:split_idx], dataset[split_idx:]  
  
# Prepare training and testing data  
X_train = [email for email, label in train_data]  
y_train = [label for email, label in train_data]  
X_test = [email for email, label in test_data]  
y_test = [label for email, label in test_data]  
  
# Initialize and train the classifier  
nb_classifier = NaiveBayesClassifier()  
nb_classifier.fit(X_train, y_train)  
  
# Evaluate the classifier  
accuracy = nb_classifier.evaluate(X_test, y_test)  
print(f"Accuracy: {accuracy:.2f}")
```

↔ Accuracy: 0.50